

23. Et, ou, non : && || !

Une vraie question de jeu se pose rarement en une seule comparaison.

1. Ce qu'il faut comprendre

« Le héros est-il dans la zone ? » veut dire : x est **au moins** 8, **et** x est **au plus** 12. Deux comparaisons, reliées par `&&` (**et**) : c'est vrai seulement si les **deux** le sont.

`||` (**ou**) est vrai si **l'une au moins** l'est : « au bord » veut dire x vaut 0 **ou** x vaut 19. `!` (**non**) retourne une réponse : `!dans` est vrai quand `dans` est faux.

Ces opérateurs **s'arrêtent dès que la réponse est connue**. Dans `x > 0 && ...`, si x vaut 0, la droite n'est même pas calculée. Ce n'est pas un détail : c'est ce qui permet d'écrire `if (x > 0 && ...)` sans jamais lire une case avant le début d'un tableau.

Regarde aussi la ligne qui déplace : `bouton(DROITE) && x < 19`. On n'avance que si la touche est enfoncée **et** qu'il reste de la place. C'est la garde du chapitre précédent : jamais de x = 255 par accident.

Ranger une condition dans une variable nommée (`dans`) rend le code lisible : `if (dans)` se lit comme une phrase.

Dernier détail : on ne réécrit l'écran **que si x a changé** (`x != ancien`). Écrire à l'écran prend du temps sur cette console — le chapitre des boucles dira exactement pourquoi.

À toi : ajoute une seconde zone, de 15 à 17, et affiche « ZONE 2 ».

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **846 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	231 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	85 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	288 o	← CopierTuiles — 18 lettres sur 43
"GAUCHE DROITE"	13 o	← texte() — un texte gravé

```

" | | "      5 o ← texte() – un texte gravé
" "          1 o ← texte() – un texte gravé
"#"          1 o ← texte() – un texte gravé
"DANS LA ZONE" 12 o ← texte() – un texte gravé
"HORS ZONE  " 12 o ← texte() – un texte gravé
"AU BORD"      7 o ← texte() – un texte gravé
" "           7 o ← texte() – un texte gravé

```

Lettres gravées : ABCDEGHILNORSTUZ#|

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,

DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte() × 8    102 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un # se déplace avec GAUCHE et DROITE sur la ligne 10.
// L'écran dit s'il est DANS la zone (colonnes 8 à 12) et s'il est AU BORD.
//
// Ce qui est nouveau : combiner des conditions.
//  && = ET   : vrai seulement si les DEUX côtés sont vrais
//  || = OU   : vrai si AU MOINS UN des deux côtés est vrai
//  !  = NON  : retourne la réponse (vrai devient faux, faux devient vrai)

#include <texte>    // écrit un texte à l'écran
#include <bouton>  // lit un bouton de la manette

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    uint8_t x = 2;                          // La colonne du #, de 0 à 19. Il part de la colonne 2.
    uint8_t ancien = 0;                     // La colonne où le # était dessiné avant.
                                           // Elle est différente de x : cela force le premier dessin.

    texte(1, 1, "GAUCHE DROITE"); // La consigne.
    texte(8, 12, "|  |");          // Dessine la zone : les barres sont en colonnes 8 et 12.

    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante.

        // Bouger, sans sortir de l'écran :
        if (bouton(DROITE) && x < 19) x++;   // DROITE tenue ET pas déjà au bord droit (19) :
                                           // x gagne 1. Si x vaut 19, rien ne se passe.
        if (bouton(GAUCHE) && x > 0) x--;    // GAUCHE tenue ET pas déjà au bord gauche (0) :
                                           // x perd 1 (« -- » retire 1).

        if (x != ancien) {                 // On ne redessine QUE si le # a bougé (x différent d'avant).
            uint8_t dans = x >= 8 && x <= 12; // 1 si x est entre 8 et 12, sinon 0.
                                           // Exemple : x = 10 → 10 >= 8 (vrai) ET 10 <= 12 (vrai) → 1.
                                           //           x = 14 → 14 <= 12 est faux → 0.
            uint8_t bord = x == 0 || x == 19; // 1 si x vaut 0 OU vaut 19, sinon 0.

            texte(ancien, 10, " "); // Efface le # à son ancienne place (un espace).
            texte(x, 10, "#");       // Le dessine à sa nouvelle place.

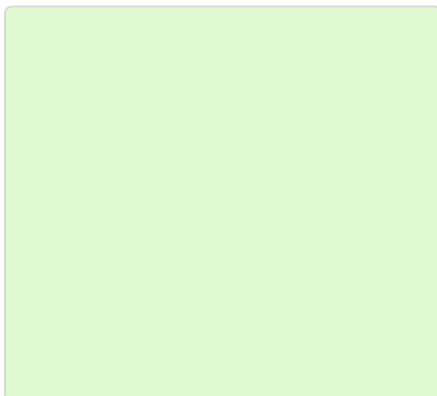
            if (dans) texte(1, 4, "DANS LA ZONE"); // Si dans vaut 1.
            if (!dans) texte(1, 4, "HORS ZONE "); // Si dans vaut 0 : « !dans » vaut 1.
                                           // Les espaces effacent la fin de l'ancien texte.

            if (bord) texte(1, 6, "AU BORD");     // Au bord : on l'écrit...
            else texte(1, 6, " ");                // ... sinon : 7 espaces pour effacer AU BORD.

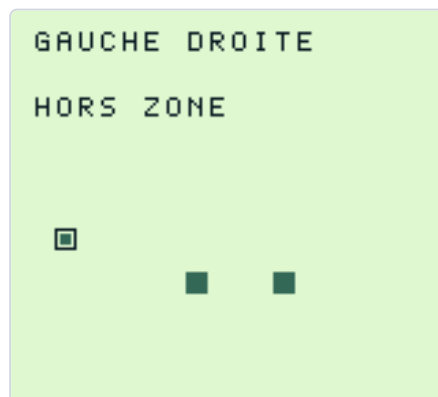
            ancien = x;                          // On retient où le # est maintenant, pour le prochain tour.
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



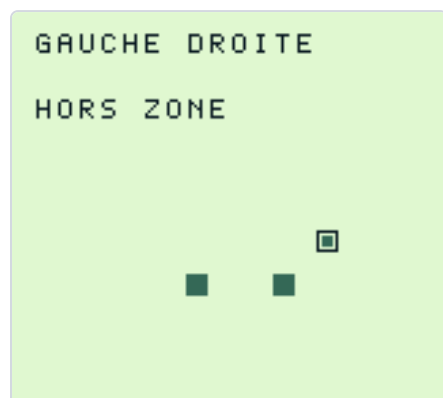
la toute première image — la console vient de démarrer — image 1



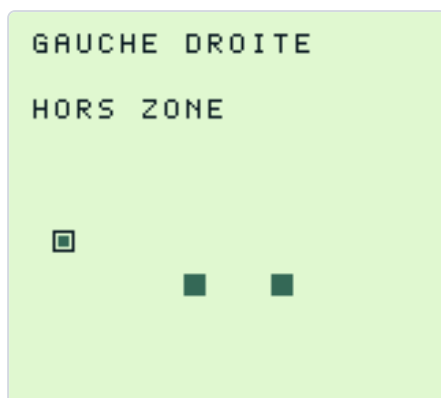
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un # qui glisse avec GAUCHE et DROITE ; « DANS LA ZONE » entre les deux barres, « AU BORD » aux extrémités.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, hors de la zone
- ✓ après quelques pas, dans la zone ($x = 9$)
- ✓ la garde && l'arrêt au bord, sans déborder ($x = 19$)
- ✓ || reconnaît le bord

Ensuite : **24. Plusieurs cas : else if, puis switch** — chapitre 4 : Les conditions : choisir. · gameboy3 — écrire une cartouche Game Boy en C++